
POLLITY

Building India's Last Mile of Digital Governance

Bootstrap MVP Plan

\$1,000 Constraint

Jan-Sunwai: Zero-to-Beta on a Shoestring

Budget:	\$1,000 total (hard ceiling)
Builder:	Piyush Zaware (CTO) — solo engineering
Date:	March 2026
Beta Goal:	50 active users, NPS > 40
Monthly Burn:	\$10–15/month (infrastructure only)
Runway:	60–80 months at infrastructure burn rate

\$1,000 is not a funding problem. **It is a prioritisation problem.** The technology to build Jan-Sunwai costs almost nothing — Supabase, Vercel, Railway, and Anthropic's Claude Haiku API together cost under \$15/month. The real constraints are **Piyush's time** and **getting 50 representatives to actually use the product**. Reserve \$800+ for user acquisition, not infrastructure.

Contents

1	What Changes at \$1,000	2
2	Revised Tech Stack: Zero-Cost Where Possible	3
1	Stack Overview	3
2	Why These Specific Choices	3
3	The \$1,000 Budget: Full Allocation	4
1	What Gets Deferred (and Why It Is Fine)	4
4	Simplified Architecture for Bootstrap	5
1	System Diagram	5
2	What Is Removed vs. Original Architecture	5
5	Simplified Repository Structure	6
6	Build Timeline: 14 Weeks Solo to Beta	8
1	Parallel Track: User Acquisition (Manav + Noorul)	8
7	The Hard Truths at \$1,000	9
8	Build Order: The Optimal Sequence	10
9	Bootstrap-Specific Risk Register	11
10	The Upgrade Path: Bootstrap to Seed Round	12

What Changes at \$1,000

Original plan assumed \$55K for product development. At \$1,000, three things become non-negotiable:

- No contractors. Piyush builds the entire stack.
- No AWS. Replace with managed free-tier platforms.
- No compliance/security spend until seed round.

Everything else stays the same. The product scope, the user target, the beta milestones — unchanged.

The counterintuitive truth: The \$1,000 plan has *longer runway* than the \$100K plan because the \$100K plan burns on salaries and AWS. Infrastructure is essentially free in 2026 if you use the right platforms. The bottleneck is never money at this stage — it is **time to ship** and **trust with political users**.

Revised Tech Stack: Zero-Cost Where Possible

1 Stack Overview

2 Why These Specific Choices

Railway over Render (free tier): Render's free tier spins down the server after 15 minutes of inactivity. A WhatsApp webhook that arrives during a cold start will time out and Meta will stop sending webhooks. Railway's \$5/month Starter keeps the FastAPI process alive 24/7. This is non-negotiable for webhook reliability.

Supabase over self-hosted Postgres: Supabase free tier gives you managed Postgres, Row Level Security (multi-tenant data isolation), built-in auth, realtime subscriptions, and daily backups — for \$0. The equivalent on AWS would cost \$30–50/month minimum.

Upstash Redis over Redis Cloud: Upstash charges per-request, not per-instance. 10,000 commands/day free is 300,000 commands/month — more than sufficient for a 50-user beta processing hundreds of grievances. Redis Cloud free tier has connection limits that become problematic.

Claude Haiku over GPT-4o-mini: Claude Haiku costs \$0.25/1M input tokens. For 50 beta users generating 50 grievances each, with average 200 tokens per classification request: $50 \times 50 \times 200 = 500,000$ tokens = **\$0.13 total for the entire beta**. NLP is effectively free at this scale.

No Terraform / no Docker required: At this scale (1 Railway service, 1 Vercel project, 1 Supabase project), infrastructure-as-code adds complexity without benefit. Platform UIs are sufficient. Docker can be added later when onboarding contractors.

The \$1,000 Budget: Full Allocation

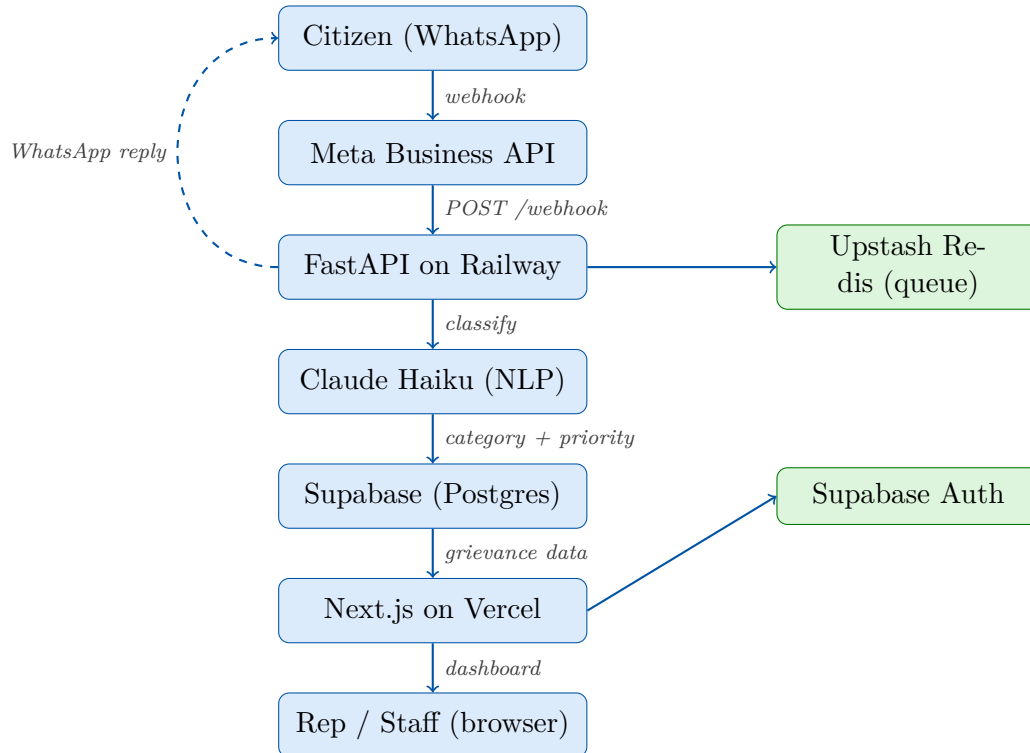
The most important insight in this document: **infrastructure costs \$7–10/month.** That is \$120 for a full year. The remaining \$880 should not go to technology. It should go to getting 50 representatives to actually use the product. No amount of engineering solves a user acquisition problem.

1 What Gets Deferred (and Why It Is Fine)

Simplified Architecture for Bootstrap

At \$1,000, the architecture removes every layer that is not essential for the core user flow.

1 System Diagram



2 What Is Removed vs. Original Architecture

Simplified Repository Structure

At \$1,000 with a solo builder, complexity is the enemy. The repo structure is simplified to remove infrastructure-as-code, separate worker services, and multi-environment config.

```
pollity-mvp/
|
+-- apps/
|   +-- web/                                # Next.js 14 frontend (Vercel)
|   |   +-- src/
|   |   |   +-- app/
|   |   |   |   +-- (auth)/                # login, register
|   |   |   |   +-- (dashboard)/          # main rep dashboard
|   |   |   |   |   +-- grievances/
|   |   |   |   |   +-- analytics/
|   |   |   |   |   +-- settings/
|   |   |   |   |   +-- onboarding/
|   |   |   |   +-- components/           # ui/, grievances/, layout/
|   |   |   |   +-- lib/                  # api.ts, auth.ts, utils.ts
|   |   |   |   +-- types/                # grievance.ts, office.ts
|   |   +-- package.json
|   |   +-- next.config.js
|   |
|   +-- api/                                # FastAPI backend (Railway)
|   |   +-- app/
|   |   |   +-- main.py                   # entry point
|   |   |   +-- core/
|   |   |   |   +-- config.py             # settings from env vars
|   |   |   |   +-- database.py          # Supabase + SQLAlchemy
|   |   |   |   +-- modules/
|   |   |   |   |   +-- jan_sunwai/
|   |   |   |   |   |   +-- router.py
|   |   |   |   |   |   +-- models.py
|   |   |   |   |   |   +-- schemas.py
|   |   |   |   |   |   +-- service.py
|   |   |   |   |   |   +-- whatsapp.py
|   |   |   |   |   +-- auth/
|   |   |   |   |   +-- offices/
|   |   |   |   +-- workers/
|   |   |   |   |   +-- classifier.py     # Claude Haiku pipeline
|   |   +-- tests/
|   |   +-- alembic/                      # DB migrations
|   |   +-- requirements.txt
|   |
+-- docs/
|   +-- architecture.md
|   +-- onboarding.md                    # new dev: running in 10 min
|   +-- data-model.md
|   +-- whatsapp-setup.md
|
+-- scripts/
|   +-- seed_db.py                      # insert test grievances
|   +-- test_webhook.py                 # simulate WhatsApp message
|
+-- .github/
|   +-- workflows/
```

```
|      +-- ci.yml                # lint + tests on every PR  
|  
+-- .env.example  
+-- .gitignore  
+-- README.md
```

What is removed from the original structure:

- `infra/terraform/` — no AWS, no Terraform needed
- `infra/docker-compose.yml` — optional for solo dev
- `deploy.yml` — Railway auto-deploys from `main` branch push
- Separate `workers/` service — background tasks stay inside FastAPI

This is **not cutting corners**. It is right-sizing architecture for the current team size (1 person) and current user base (0 to 50 people).

Build Timeline: 14 Weeks Solo to Beta

Piyush builds this alone. The timeline below is calibrated assuming 20–25 hours/week of coding time (evenings and weekends around academic responsibilities).

1 Parallel Track: User Acquisition (Manav + Noorul)

While Piyush builds, Manav and Noorul run the user pipeline. The product launches into a prepared audience, not into a void.

The Hard Truths at \$1,000

Hard Truth 1: You cannot hire anyone. At \$1,000, there is no budget for contractors, freelancers, or part-time developers. Piyush must build the entire stack: FastAPI backend, Next.js frontend, WhatsApp integration, Claude API pipeline, Supabase schema, and deployment. This is achievable in 14 weeks at 20–25 hours/week — but only if scope is ruthlessly controlled. Every feature request that is not Jan-Sunwai v1 must be rejected until beta.

Hard Truth 2: DPDP compliance is deferred. The original plan calls for India-based servers (AWS Mumbai) for DPDP Act compliance. At \$1,000, the backend runs on Railway (US/EU servers). This is a legal grey area for a pre-seed startup with 50 users. The pragmatic answer: DPDP enforcement rules are not yet finalised, and no regulator will pursue a zero-revenue startup with 50 users. Migrate to AWS Mumbai at the Seed round. **Document the decision and the plan to fix it.** Do not pretend the risk does not exist — acknowledge it in the beta user agreements.

Hard Truth 3: The biggest risk is not technical. The tech will work. Claude Haiku classifies Hindi grievances. WhatsApp webhooks are reliable. Supabase handles the load. The existential risk is whether 50 elected representatives will actually change their workflow to use a new tool — even a free one. Political users are resistant to change. Trust takes time. The \$600 in the user acquisition budget is more important than the \$120 in infrastructure budget.

Hard Truth 4: The NPS target requires product quality, not features. NPS > 40 from 50 political users means the product must be genuinely useful, not just functional. This means: the WhatsApp auto-reply must be in the constituent's language and sound human. The dashboard must load fast on a 4G mobile browser. The classification must be accurate enough that the rep trusts it. Speed and accuracy of these three things matter more than any additional feature.

Build Order: The Optimal Sequence

At \$1,000 with one builder, sequencing matters. The wrong order wastes weeks on infrastructure that breaks when the real bottleneck is elsewhere.

Bootstrap-Specific Risk Register

The Upgrade Path: Bootstrap to Seed Round

The \$1,000 bootstrap plan is not the destination — it is the validation engine. Once 50 users are active and $NPS > 40$, the seed round (\$300–500K) is unlocked. Here is exactly what changes when seed capital arrives.

The sequence that raises the seed round:

1. Ship Jan-Sunwai v1 on Railway + Supabase + Vercel.
2. Onboard 50 representatives in Delhi and Maharashtra.
3. Collect: $NPS > 40$, 1,000+ grievances processed, 5 case studies, $DAU/MAU > 0.4$.
4. Show these numbers to investors.
5. Raise \$300–500K seed.
6. Migrate to AWS Mumbai, hire first engineer, begin Vidhayak-Mitra.

The bootstrap plan is not a compromise. It is the proof that the team can build and ship without needing money — which is the single most credible signal to a pre-seed investor.

Table 1: Original Plan vs. \$1,000 Bootstrap Plan

Dimension	Original (\$100K)	Bootstrap (\$1,000)
Infrastructure	AWS Mumbai (ap-south-1), \$12K/yr	Supabase + Vercel + Railway, \$10–15/mo
Backend hosting	ECS Fargate + ALB	Railway (\$5/mo, always-on)
Database	RDS Postgres + managed backups	Supabase free tier (500MB, daily backups)
Frontend hosting	AWS S3 + CloudFront	Vercel free tier
Redis / Queue	ElastiCache (\$50+/mo)	Upstash Redis free tier (10K/day)
NLP	Claude API + fine-tuning budget	Claude Haiku API (\$0.25/1M tokens)
Contractors	\$28K (6-week sprint)	None — CTO builds solo
Security audit	\$4,600	Deferred to seed round
DPDP compliance setup	\$3K legal	Deferred to seed round
ISO 27001 assessment	Part of \$15K ops budget	Deferred to seed round
Terraform / IaC	Full AWS infrastructure code	Platform UIs (no Terraform needed)
Docker (local dev)	Required for dev parity	Optional — run locally with <code>venv</code>
Monthly burn	\$3,000–5,000	\$10–15
Runway	20 months (at \$100K)	60–80 months

Table 2: Bootstrap Tech Stack with Monthly Costs

Layer	Service	Free Tier / Notes	Monthly Cost
Backend	FastAPI on Railway	512MB RAM, always-on at \$5/mo Starter plan	\$5
Frontend	Next.js on Vercel	Unlimited personal projects, 100GB bandwidth	\$0
Database	Supabase (free tier)	500MB Postgres, 50K MAU auth, Row Level Security	\$0
Redis Queue	/ Upstash Redis	10,000 commands/day free — enough for 50 beta users	\$0
WhatsApp	Meta Business API	First 1,000 conversations/month free	\$0
NLP	Anthropic Claude Haiku	\$0.25/1M input tokens. 50 users × 50 grievances = ~\$0.10 total	~\$2–5
Auth	Supabase Auth	Included in free tier	\$0
File storage	Supabase Storage	1GB included in free tier	\$0
CI/CD	GitHub Actions	2,000 min/month free	\$0
Error tracking	Sentry free tier	5K errors/month, 14-day retention	\$0
Analytics	PostHog free tier	1M events/month, session recording	\$0
Email (digest)	Resend free tier	3,000 emails/month free	\$0
Domain	pollity.in	One-time annual renewal	\$15/yr
Total monthly infrastructure cost			\$7–10

Table 3: Full \$1,000 Budget Allocation

Category	Budget	What It Buys
Infrastructure (12 months)	\$120	Railway \$5/mo $\times$ 12 + Anthropic credits + domain renewal
User Acquisition	\$600	Train tickets Delhi/Mumbai, coffee meetings with MLAs and councillors, 2–3 field trips to constituency offices, printing of on-boarding guides in Hindi
WhatsApp Business Setup	\$50	Meta Business verification, test message volume during development
Contingency	\$230	Unexpected API overages, extra travel, emergency tooling
Total	\$1,000	

Table 4: Deferred Items and When to Revisit

Deferred Item	Revisit When	Why It Is Fine to Defer
AWS Mumbai migration (DPDP)	Seed round (\$300K+)	DPDP enforcement rules are not yet final. No regulator will target a 50-user pre-seed startup. Migrate before commercial launch.
ISO 27001 assessment	Series A	Required for enterprise/government contracts, not for beta.
Penetration test / security audit	Seed round	With 50 beta users, attack surface is minimal. Basic OWASP checklist is sufficient.
Contractor engineering support	Seed round	CTO can build the full Jan-Sunwai v1 solo in 14–16 weeks.
Docker / Terraform / IaC	When first contractor joins	Adds 2 weeks of setup time with zero user-facing benefit.
Multi-language NLP fine-tuning	After beta data collected	Need real grievance data first. Generic Claude Haiku handles Hindi/Marathi adequately for beta.

Component	Status	Replacement
AWS ECS / EC2	Removed	Railway (\$5/mo)
AWS RDS	Removed	Supabase free Postgres
AWS ElastiCache	Removed	Upstash free Redis
AWS CloudFront + S3	Removed	Vercel free CDN
Terraform / IaC	Removed	Platform UIs
Docker (local dev)	Optional	Local Python venv + Supabase local CLI
Separate worker service	Simplified	Background tasks handled within FastAPI using <code>BackgroundTasks</code>
Full CI/CD deploy pipeline	Simplified	GitHub Actions: test only; Railway auto-deploys from <code>main</code> branch

Table 5: 14-Week Solo Build Timeline

Weeks	Phase	Deliverables
1–2	Foundation	Supabase project setup + DB schema. Railway FastAPI skeleton. Vercel Next.js skeleton with auth. WhatsApp Business API application submitted to Meta. GitHub Actions CI. <code>.env.example</code> complete.
3–5	WhatsApp Core	Webhook receiver with HMAC verification. Message parsed and queued via Upstash. Claude Haiku classification pipeline (language + category + priority). Grievance written to Supabase. Auto-acknowledgement sent to citizen.
6–9	Dashboard	Rep dashboard: grievance list, filters (status/category/date), detail view. Status management (Open → In Progress → Resolved → Closed). Status change triggers WhatsApp notification to citizen. Staff invite + role system. Basic rep onboarding flow.
10–11	Polish	Bulk actions. Analytics page (volume, resolution time, category breakdown). Mobile-responsive layout. Weekly email digest (Resend). CSV export.
12–13	Beta Prep	Security basics (OWASP Top 10 self-audit, auth token review). DPDP-lite: data retention policy, consent message to citizens. In-app NPS via PostHog. Onboarding guide PDF in English + Hindi. Load test: simulate 200 concurrent messages.
14	Beta Launch	Onboard first 10 representative offices in Delhi (personal network via Noorul). Monitor, fix bugs. Week 14 = real users, real grievances, real feedback.

Table 6: 14-Week User Acquisition Track

Weeks	Activities
1–4	Map personal network: list every MLA, councillor, and panchayat leader known directly or one degree away. Target: 20 warm contacts.
5–8	First meetings: coffee + demo of prototype (even basic). Collect pain points. Ask: “Would you use a WhatsApp system that categorises and tracks grievances for you?” Get verbal commitments.
9–12	Sign up 10 rep offices for beta. Get one staff member per office committed as the primary user. Collect their WhatsApp number for the business API.
13–14	Onboard beta users. Personally sit with each office for 1 hour. Show them the dashboard. Send first test grievance together.
15+	Expand to 50 offices. Weekly check-ins. Collect NPS. Document 5 case studies.

Table 7: Optimal Build Sequence with Rationale

#	Build This	Why This Order
1	Supabase schema: offices , grievances , messages , staff	Everything else depends on the data model. Get this right first — changing it later is painful.
2	FastAPI webhook endpoint: receive Meta POST, verify HMAC signature, write raw message to messages table	This is the one piece that can break silently. Test it with real WhatsApp messages before building anything else on top.
3	Claude Haiku classifier: takes raw message, returns language + category + priority as JSON	This is the core AI value-add. Validate accuracy on 20–30 real Hindi/Marathi test messages before wiring it into the flow.
4	Auto-acknowledgement: send WhatsApp reply to citizen confirming receipt in their language	Validates the outbound API direction. Lets you demo a full loop (citizen sends → citizen gets reply) before the dashboard exists.
5	Supabase Auth + multi-tenant isolation: rep office login, RLS policies	Must exist before the dashboard. Do not skip RLS — data isolation is a trust requirement, not a nice-to-have.
6	Next.js dashboard: grievance list + detail view + status update	First thing a rep actually sees. Polish this page above all others — it determines NPS.
7	Status change → WhatsApp notification	Closes the loop for the citizen. This is the feature that

Table 8: Risks Specific to the \$1,000 Bootstrap Scenario

Risk	Prob.	Impact	Mitigation
Piyush burns out or runs out of time	High	Critical	Enforce ruthless scope. Jan-Sunwai v1 only. If academic workload spikes, push beta by 4 weeks — do not try to build faster.
Supabase free tier hits limits	Low	Medium	500MB is ample for 50 beta users with 1,000 grievances. Monitor storage weekly. Upgrade to Pro (\$25/mo) only if hit.
Railway goes down mid-webhook	Low	High	Set up UptimeRobot (free) to ping the health endpoint every 5 min. WhatsApp retries failed webhooks for 7 days — most messages will be recovered.
Meta rejects WhatsApp Business API	Medium	Critical	Apply immediately. Use Twilio WhatsApp sandbox as fallback for dev only. Have a backup plan: Telegram Bot API (same architecture, instant approval).
No one uses the product	High	Critical	The biggest risk. Mitigation: Noorul personally onboards the first 10 rep offices. Do not launch to cold contacts.
Claude Haiku misclassifies Hindi	Medium	Medium	Add human-review queue for confidence < 0.7 . In beta, low accuracy is a learning opportunity, not a failure.
Rep staff don't check dashboard	High	High	Make the WhatsApp flow standalone — staff don't need to check the dashboard for the system to function. Dashboard is a bonus.

Table 9: Bootstrap to Seed: What Gets Upgraded

Component	Bootstrap (\$1K)	Post-Seed (\$300K+)
Backend hosting	Railway \$5/mo	AWS ECS Fargate, ap-south-1
Database	Supabase free	AWS RDS Postgres, Mumbai
Data residency	US/EU (Railway)	India (DPDP compliant)
Redis	Upstash free tier	AWS ElastiCache
Infrastructure	Platform UIs	Terraform IaC
Engineering	CTO solo	1 senior full-stack hire
NLP	Generic Haiku Claude	Fine-tuned on beta grievance data
Security	OWASP self-audit	External pentest + ISO 27001
Compliance	Deferred	DPDP setup + legal counsel